

MARVEL 论文深度解析与学习心得

论文标题: MARVEL: Multi-Agent Reinforcement Learning for Constrained Field-of-View Multi-Robot Exploration in Large-Scale Environments

发表会议: ICRA 2025

作者团队: Jimmy Chiun, Shizhe Zhang, Yizhuo Wang, Yuhong Cao, Guillaume Sartoretti

论文来源: [ICRA-2025_MARVEL_Multi-Agent_Reinforcement_Learning_for_Constrained_Field-of-View_Multi-Robot_Exploration_in_Large-Scale_Environments](file:///C:/Users/Lenovo/Desktop/MARVEL-main/ICRA-2025_MARVEL_Multi-Agent_Reinforcement_Learning_for_Constrained_Field-of-View_Multi-Robot_Exploration_in_Large-Scale_Environments.pdf)

一、研究背景与问题定义

1.1 核心挑战：受限视场下的多机器人探索

传统多机器人探索方法普遍假设使用全向传感器（如 LiDAR），但实际场景中存在大量约束：

约束类型	具体表现	带来的挑战
传感器限制	无人机载荷受限，只能使用轻量级方向性传感器（如相机）	需要同时规划位置和传感器朝向
视场约束	传感器视场角有限（如 120° ）	前沿检测频率降低，奖励信号稀疏
环境规模	大规模室内环境（可达 $90m \times 90m$ ）	动作空间庞大，计算复杂度高
多智能体协作	团队奖励需合理归因到单个智能体	信用分配困难

1.2 问题形式化定义

环境建模: 环境 E 为有界占据栅格地图，分为自由区域 E_f 和占据区域 E_o 。

信念状态: 每个智能体维护三元组 $B_i = \{B_f, B_o, B_u\}$ （自由/占据/未知区域）。

探索目标: 最大化探索率 $\rho = \frac{B_f}{E_f}$ ，当 $\rho \geq 0.99$ 时任务完成。

1.3 现有方法的局限性

论文系统地分析了传统方法的不足：

前沿-based 方法:

- Nearest 方法：短视策略导致重复访问已探索区域
- MMPF：局部探索为主，缺乏全局协调能力
- NBVP：采样效率随环境规模下降，计算成本高

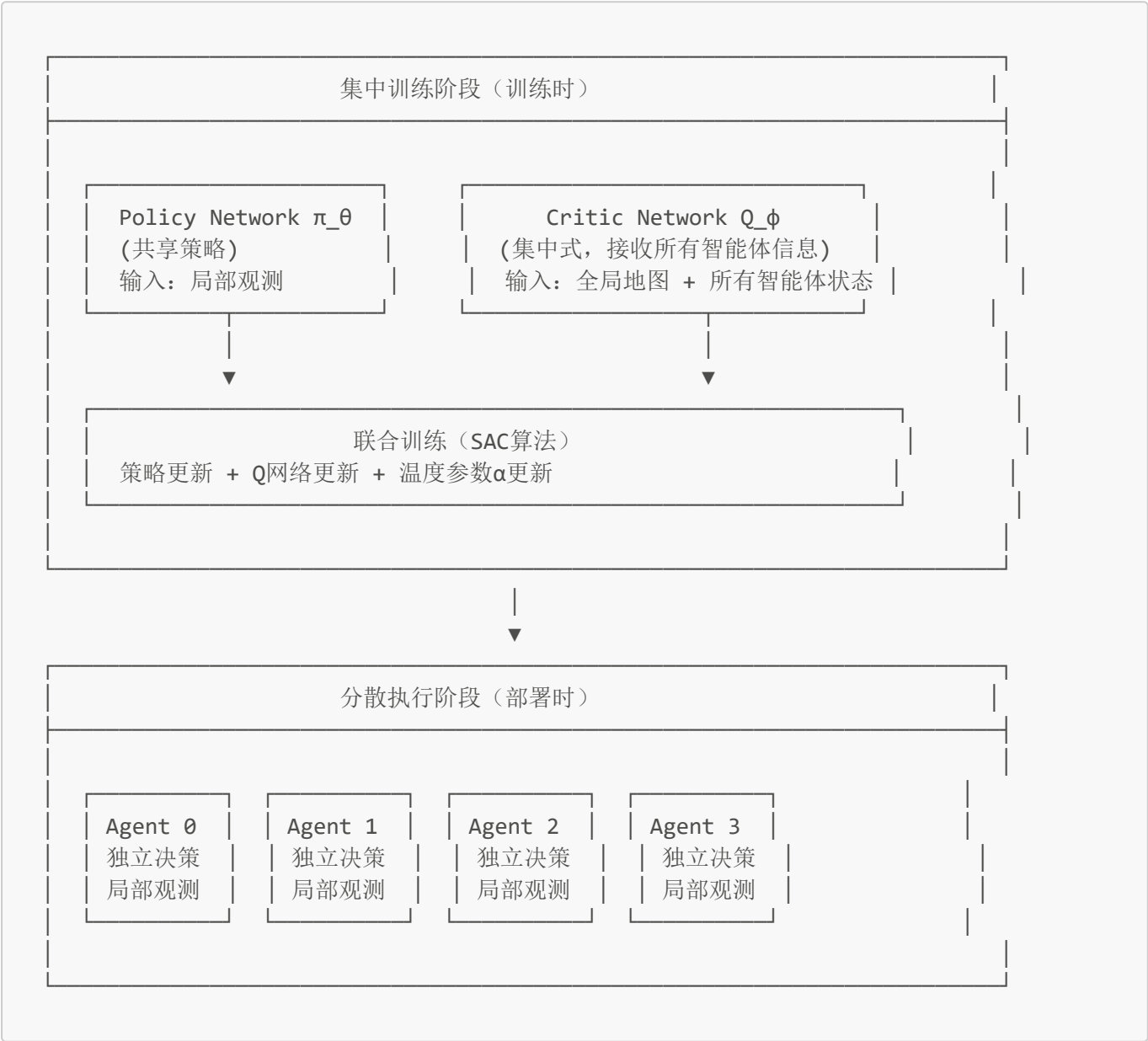
基于学习的方法:

- 受限视场使动作空间更庞大
- 传感器约束导致稀疏奖励问题
- 多智能体信用分配困难
- 时空短视问题（难以平衡当前视角利用与长期探索）

二、MARVEL 核心算法架构

2.1 整体框架：CTDE 范式

MARVEL 采用 **集中训练、分散执行（CTDE）** 范式：



2.2 图结构表示：环境建模创新

MARVEL 的核心创新之一是将环境表示为**碰撞-free 图**：

图构建流程：

环境栅格地图 → 均匀采样可行视点 V_t → 连接碰撞-free边 E_t → 构建图 $G_t = (V_t, E_t)$

节点特征设计（6 维，见 [node_manager.py](file:///C:/Users/Lenovo/Desktop/MARVEL-main/utils/node_manager.py))：

属性	符号	含义	数据类型
相对位置	$(\Delta x_{ik}, \Delta y_{ik})$	节点相对于智能体 i 的坐标	连续值
效用值	u_k	节点处可见前沿数量（归一化）	连续值
智能体占用	o_k	是否被当前/其他智能体占用	三元信号
路标信号	g_k	是否在到最近前沿的 A* 路径上	二进制
信息性朝向	h_k	最大化可见前沿的方向（归一化）	连续值

前沿分布特征：

- 每个节点周围采样 **36 个均匀分布的视场方向**
- 记录每个方向内可观测前沿的归一化值
- 形成 **36 维前沿分布向量**，捕捉空间分布信息

2.3 策略网络架构

策略网络由**编码器-解码器**结构组成（见 [model.py](file:///C:/Users/Lenovo/Desktop/MARVEL-main/utils/model.py#L198-L283))：

编码器（6层堆叠图注意力）：

```
# 节点特征编码
node_feature = self.initial_embedding(node_inputs) # 6维 → 128维

# 6层堆叠图注意力（图掩码限制注意力范围）
enhanced_node_feature = self.encoder(src=node_feature,
                                     key_padding_mask=node_padding_mask,
                                     attn_mask=edge_mask)

# 前沿分布特征融合（Conv1d编码）
frontier_distribution = frontier_distribution.permute(0, 2, 1)
frontiers_feature = self.frontiers_embedding(frontier_distribution) # 36维 → 128维

# 拼接融合
enhanced_node_feature = self.node_frontiers_embedding(
    torch.cat((enhanced_node_feature, frontiers_feature), dim=-1))
```

解码器（单头注意力指针网络）：

- 输入：当前节点特征 + 邻居节点特征（增强朝向信息）

- 输出：动作概率分布（指向邻居节点-朝向对）
- 使用 **指针网络（Pointer Network）** 动态适应不同数量的邻居节点

2.4 Critic 网络架构

Critic 网络采用**特权学习（Privileged Learning）**机制（见 [model.py] (file:///C:/Users/Lenovo/Desktop/MARVEL-main/utils/model.py#L286-L392)）：

特权学习设计：

- **训练时**：接收完整 Ground Truth 全局地图和完整图信息
- **执行时**：仅使用局部观测
- 增强信用分配能力，提高训练稳定性

多智能体注意力机制（MAAC）：

```
# 获取其他智能体的当前/下一节点特征
all_agent_node_feature = torch.gather(enhanced_node_feature, 1,
                                     all_agent_indices.repeat(1, 1, embedding_dim))
all_agent_selected_neighboring_feature = torch.gather(enhanced_node_feature, 1,
all_agent_next_indices.repeat(1, 1, embedding_dim))

# 融合其他智能体动作信息
all_agent_action_features = torch.cat((all_agent_node_feature,
all_agent_selected_neighboring_feature), dim=-1)
all_agent_action_features = self.all_agent_embedding(all_agent_action_features)

# 多智能体注意力解码
global_state_action_feature, _ = self.agent_decoder(current_state_feature,
all_agent_action_features, agent_mask)
```

2.5 训练算法：Soft Actor-Critic (SAC)

MARVEL 使用 **SAC（最大熵强化学习）** 作为训练算法（见 [driver.py] (file:///C:/Users/Lenovo/Desktop/MARVEL-main/driver.py#L251-L303)）：

目标函数：

$$J(\pi) = \mathbb{E}[Q(s, a) - \alpha \cdot \log \pi(a|s)]$$

训练流程：

步骤	操作	频率	核心代码位置
策略更新	最大化 $J(\pi)$	每步	[driver.py#L257-L266](file:///C:/Users/Lenovo/Desktop/MARVEL-main/driver.py#L257-L266)
Q网络更新	最小化 MSE 损失	每步	[driver.py#L278-L296](file:///C:/Users/Lenovo/Desktop/MARVEL-main/driver.py#L278-L296)

步骤	操作	频率	核心代码位置
温度参数更新	自适应调整 α	每步	[driver.py#L298-L303](file:///C:/Users/Lenovo/Desktop/MARVEL-main/driver.py#L298-L303)
目标Q网络更新	同步权重	每 64 次更新	[driver.py#L334-L340](file:///C:/Users/Lenovo/Desktop/MARVEL-main/driver.py#L334-L340)

关键训练参数：

参数	值	说明
折扣因子 γ	1	无折扣（探索任务特性）
批次大小	256	训练批次
回放缓冲区	10,000	经验池大小
学习率	10^{-5}	Adam 优化器
目标熵	$0.01 \cdot \log(k)$	最大熵目标

三、关键改进措施详解

3.1 改进一：图注意力网络 —— 显式建模空间关系

问题：传统方法难以建模复杂空间关系和大规模环境

解决方案：6层堆叠图注意力网络

技术要点：

- 使用图掩码 M_t （邻接矩阵）限制每层注意力范围
- 通过多层堆叠逐步聚合非邻居信息，实现长距离依赖建模
- 节点特征从6维增强到128维，提升表达能力

代码实现 ([model.py#L176-L185](file:///C:/Users/Lenovo/Desktop/MARVEL-main/utils/model.py#L176-L185))：

```
class Encoder(nn.Module):
    def __init__(self, embedding_dim=128, n_head=4, n_layer=6):
        super(Encoder, self).__init__()
        self.layers = nn.ModuleList(EncoderLayer(embedding_dim, n_head) for i in range(n_layer))

    def forward(self, src, key_padding_mask=None, attn_mask=None):
        for layer in self.layers:
            src = layer(src, key_padding_mask=key_padding_mask, attn_mask=attn_mask)
        return src
```

对比优势：

维度	传统方法	MARVEL
空间关系	隐式学习	显式建模（图注意力）
环境规模	局限于小规模	支持90m×90m大规模环境
拓扑复杂性	处理能力有限	有效处理复杂拓扑

3.2 改进二：前沿-朝向特征融合 —— 增强环境理解

问题：Learnt-Greedy 方法在处理小分散前沿时表现不佳

论文对比描述：

"learnt-greedy methods delivers faster coverage of 90% of the environment than nearest and MMPF... However, this method struggles with small scattered frontiers, produced due to suboptimal viewpoint orientations, leading to repeated revisits and longer trajectories."

解决方案：智能融合前沿分布 + 朝向特征

技术要点：

- 前沿分布特征：36维向量，编码每个方向的前沿可见度
- 朝向特征：
 - FoV内朝向二进制向量（36维）
 - 历史朝向访问记录（36维）
- 融合方式：Conv1d 编码前沿分布 + FFN 编码朝向信息

代码实现 ([model.py#L206-L233](file:///C:/Users/Lenovo/Desktop/MARVEL-main/utils/model.py#L206-L233))：

```
# 前沿分布编码 (Conv1d)
self.frontiers_embedding = nn.Conv1d(num_angles_bin, embedding_dim, kernel_size=3,
padding=1)

# 朝向特征编码
self.best_headings_embedding = nn.Linear(num_angles_bin, embedding_dim)
self.visited_headings_embedding = nn.Linear(num_angles_bin, embedding_dim)

# 融合节点特征与前沿分布
enhanced_node_feature = self.node_frontiers_embedding(
    torch.cat((enhanced_node_feature, frontiers_feature), dim=-1))
```

实验对比：

指标	Learnt-Greedy	MARVEL	提升
轨迹长度（4智能体）	433.03	357.5	14.2%

指标	Learnt-Greedy	MARVEL	提升
90%覆盖	318.46	294.21	7.6%
成功率	92%	100%	+8%

3.3 改进三：信息驱动动作剪枝 —— 处理大动作空间

问题：受限视场下动作空间过大（位置+朝向联合决策）

解决方案：基于信息增益的动态动作剪枝

技术要点：

- 对每个相邻节点，基于可见前沿数量选择 **3个最佳朝向**
- 形成联合动作对 (v_j, a_q) ，其中 $a_q \in \{a_1, a_2, a_3\}$
- 无前沿可见时，使用 A* 路径采样朝向

代码实现 ([agent.py#L264-L319](file:///C:/Users/Lenovo/Desktop/MARVEL-main/utils/agent.py#L264-L319)):

```
def compute_best_heading(self, node_coords, frontier_distribution,
neighbor_nodes):
    neighbor_best_headings = []
    for neighbor in neighbor_nodes:
        node_index = neighbor.item()
        heading_candidates = torch.zeros(self.num_heading_candidates,
self.num_angles_bin)

        if node_data.utility > 0:
            # 基于前沿分布选择最佳朝向
            half_fov_size = int((self.fov / 360)*self.num_angles_bin/2)
            window = np.concatenate((node_frontier_distribution[-half_fov_size:],
node_frontier_distribution,
node_frontier_distribution[:half_fov_size]))
            sum_vector = np.sum(np.take(window, indices.reshape(-1, 1) +
np.arange(-half_fov_size, half_fov_size +
1)), axis=1)
            top_n_indices = np.argsort(-sum_vector)[:self.num_heading_candidates]
        else:
            # 基于A*路径选择朝向
            angle = np.degrees(np.arctan2(next_coords[1] - coords[1],
next_coords[0] - coords[0]) % (2 * np.pi))
            new_index = int(angle / 360 * self.num_angles_bin) %
self.num_angles_bin

            neighbor_best_headings.append(heading_candidates)
```

对比优势：

维度	传统方法	MARVEL
动作空间	固定且庞大	动态剪枝（缩小3倍以上）
计算效率	随节点数线性增长	保持恒定（0.2s/决策）
决策质量	可能选择无信息量动作	仅选择高信息动作

3.4 改进四：特权学习 Critic —— 增强信用分配

问题：多智能体信用分配困难

解决方案：训练时使用 Ground Truth 全局信息

技术要点：

- **训练时**：接收完整全局地图和完整图信息
- **执行时**：仅使用局部观测
- 利用全局信息辅助信用分配，提高训练稳定性

代码实现 ([driver.py#L205-L250](file:///C:/Users/Lenovo/Desktop/MARVEL-main/driver.py#L205-L250))：

```
if TRAIN_ALGO in (2, 3):
    # 使用Ground Truth信息训练Critic
    gt_node_inputs = torch.stack(rollouts[19]).to(device)
    gt_node_padding_mask = torch.stack(rollouts[20]).to(device)
    gt_edge_mask = torch.stack(rollouts[21]).to(device)
    # ... 其他Ground Truth特征

    state = [gt_node_inputs, gt_node_padding_mask, gt_edge_mask, ...]
    next_state = [gt_next_node_inputs, gt_next_node_padding_mask,
gt_next_edge_mask, ...]
```

对比优势：

维度	传统方法	MARVEL
信用分配	困难（局部信息有限）	增强（全局信息辅助）
训练稳定性	可能不稳定	更稳定（特权学习）
协作能力	有限	更强（MAAC架构）

3.5 改进五：MAAC 多智能体注意力 —— 增强团队协作

问题：智能体间缺乏有效协调机制

解决方案：基于 MAAC（Multi-Agent Actor-Attention-Critic）的隐式通信

技术要点：

- Critic 融合其他智能体的当前/下一节点特征作为 key-value pairs

- 增强当前智能体的决策能力
- 实现隐式通信，促进团队协作

代码实现 ([model.py#L368-L383](file:///C:/Users/Lenovo/Desktop/MARVEL-main/utils/model.py#L368-L383)):

```
if all_agent_indices != None:
    # 获取所有智能体的动作信息
    all_agent_node_feature = torch.gather(enhanced_node_feature, 1,
                                         all_agent_indices.repeat(1, 1,
embedding_dim))
    all_agent_selected_neighboring_feature = torch.gather(enhanced_node_feature,
1,

all_agent_next_indices.repeat(1, 1, embedding_dim))

    # 融合为全局状态-动作特征
    all_agent_action_features = torch.cat((all_agent_node_feature,
all_agent_selected_neighboring_feature), dim=-1)
    all_agent_action_features =
self.all_agent_embedding(all_agent_action_features)

    # 多智能体注意力解码
    global_state_action_feature, _ = self.agent_decoder(current_state_feature,
all_agent_action_features, agent_mask)

    # 计算Q值
    action_features = torch.cat((current_state_feature.repeat(1,
enhanced_neighbor_features.size()[1], 1),
                                enhanced_neighbor_features,
                                global_state_action_feature.repeat(1,
enhanced_neighbor_features.size()[1], 1)), dim=-1)
    q_values = self.q_values_layer(action_features)
```

对比优势:

维度	传统方法	MARVEL
智能体通信	显式或无	隐式（通过Critic）
团队协作	有限	更强
重叠率	较高（Nearest: 0.693）	较低（0.170）

四、从论文中学到的关键技术经验

4.1 算法设计思想

1. 将环境建模为图结构

- 栅格地图 → 采样可行视点 → 构建碰撞-free图

- 图结构天然适合表示空间关系和拓扑信息
- 图注意力网络能够显式建模节点间的依赖关系

2. 特征工程的重要性

- 前沿分布特征：捕捉空间分布信息
- 朝向特征：解决受限视场下的视角规划问题
- 路标信号：引导智能体朝向目标区域

3. 动作空间优化

- 信息驱动剪枝：聚焦有信息量的动作
- 动态动作空间：根据当前状态调整可用动作
- 降低探索难度，提高样本效率

4. 多智能体协作机制

- 集中式 Critic：利用全局信息进行信用分配
- 隐式通信：通过共享策略和Critic实现协作
- 团队奖励：促进整体最优行为

4.2 工程实现经验

1. 分布式训练架构

- 使用 Ray 进行分布式经验收集
- 多个 meta-agent 并行运行环境
- 主进程负责网络更新和模型同步

2. 数据管理策略

- 多层经验缓冲区（40层）
- 动态调整缓冲区大小
- 支持大规模训练数据管理

3. 数值稳定性处理

- Gradient clipping（梯度裁剪）
- 双Q网络防止过估计
- 自适应温度参数平衡探索与利用

4. 环境组件模块化

- 传感器模块：独立模拟受限视场
- 节点管理：维护图结构和邻居关系
- 智能体模块：处理个体决策和状态管理

4.3 实验设计方法论

1. 对比实验设计

- 传统方法对比：Nearest, MMPF, NBVP

- 学习方法对比：Learnt-Greedy
- 消融实验：验证各组件的有效性

2. 性能指标设计

- 轨迹长度：衡量探索效率
- 覆盖速度：衡量任务完成速度
- 重叠率：衡量协作效率
- 成功率：衡量算法鲁棒性

3. 泛化能力验证

- 不同视场角（90°/120°/180°）
- 不同传感器范围（10m/12m/15m）
- 不同智能体数量（2/4/8）

4. 真实硬件验证

- 在 Crazyflie 2.1 无人机上进行实机测试
- 验证算法在真实场景中的可行性

五、实验结果与分析

5.1 性能对比（论文 Table I）

4智能体场景（FoV=120°, 范围=10m）:

指标	Nearest	MMPF	NBVP	Learnt-Greedy	MARVEL
轨迹长度 ↓	417.92	427.30	416.50	433.03	357.5
90%覆盖 ↓	346.50	358.56	329.75	318.46	294.21
重叠率 ↓	0.693	0.075	0.246	0.191	0.170
成功率 ↑	99%	95%	100%	92%	100%

关键结论：

- MARVEL 在轨迹长度上比最佳基线 NBVP 提升 **14.2%**
- 成功率达到 **100%**，表现出卓越的鲁棒性
- 重叠率较低，说明智能体协作效率高

5.2 适配性验证（论文 Table II/III）

不同视场角（4智能体，范围=10m）:

FoV	轨迹长度	90%覆盖	成功率
90°	378.99	313.67	100%
120°	357.5	294.21	100%

FoV	轨迹长度	90%覆盖	成功率
180°	328.36	273.18	100%

不同传感器范围（4智能体，FoV=120°）：

范围	轨迹长度	90%覆盖	重叠率
10m	357.5	294.21	0.170
12m	299.57	247.93	0.223
15m	258.35	212.85	0.208

结论：MARVEL 无需重新训练即可适配不同传感器配置，表现出良好的泛化能力。

六、核心创新点总结

6.1 技术贡献矩阵

创新点	论文位置	技术细节	核心价值
图注意力网络	Section IV.B	6层堆叠图注意力，基于邻接矩阵掩码	增强空间关系建模
前沿-朝向特征融合	Section IV.B	Conv1d 编码前沿分布 + FFN 编码朝向信息	提高环境理解
信息驱动动作剪枝	Section IV.A	基于可见前沿数量选择最佳朝向，减少动作空间	解决大动作空间问题
特权学习 Critic	Section IV.C	训练时使用全局信息，增强信用分配	提高训练稳定性
MAAC 架构	Section IV.C	融合其他智能体动作信息的集中式 Critic	增强多智能体协作

6.2 算法优势总结

维度	MARVEL	传统方法
视场约束	显式建模受限 FoV	假设全向传感器
动作空间	动态剪枝（3个朝向/节点）	固定离散/连续
空间关系	图注意力显式建模	隐式学习
智能体协作	集中式 Critic + 团队奖励	独立决策或显式通信
泛化能力	无需重新训练适配不同配置	需重新调参

七、学习心得与启示

7.1 核心思想启示

1. 问题分解的重要性

- 将复杂的多机器人探索问题分解为：环境建模、动作选择、多智能体协调
- 每个子问题针对性设计解决方案

2. 领域知识与深度学习的结合

- 图结构建模基于路径规划领域知识
- 前沿分布特征基于探索理论
- 动作剪枝基于信息论原理

3. 强化学习与传统方法的融合

- 使用 A* 路径规划作为启发式引导
- 使用图结构表示环境，结合注意力机制
- 传统方法提供先验知识，深度学习优化策略

7.2 工程实践启示

1. 模块化设计

- 将环境、智能体、网络、训练器分离
- 便于调试和扩展
- 支持不同场景的快速适配

2. 分布式训练

- 利用多GPU加速训练
- 多环境并行收集经验
- 提高训练效率和样本多样性

3. 实验验证的严谨性

- 全面的对比实验
- 消融实验验证组件有效性
- 真实硬件验证算法可行性

7.3 未来研究方向

1. 3D 环境扩展

- 集成高度信息和相关的 FoV 变化
- 采用稀疏图表示管理计算需求

2. 动态环境适应

- 处理动态障碍物
- 在线学习和策略调整

3. 通信约束

- 考虑有限通信范围
- 设计高效的分布式算法

4. 多任务学习

- 同时优化探索和其他任务（如目标跟踪）
- 迁移学习跨环境泛化

八、代码实现要点总结

8.1 网络架构对应

组件	代码位置	说明
PolicyNet	[model.py#L198-L283] (file:///C:/Users/Lenovo/Desktop/MARVEL-main/utils/model.py#L198-L283)	策略网络，包含编码器和解码器
QNet	[model.py#L286-L392] (file:///C:/Users/Lenovo/Desktop/MARVEL-main/utils/model.py#L286-L392)	Q值网络，支持MAAC模式
MultiHeadAttention	[model.py#L48-L119] (file:///C:/Users/Lenovo/Desktop/MARVEL-main/utils/model.py#L48-L119)	图注意力，用于节点特征增强
SingleHeadAttention	[model.py#L5-L46](file:///C:/Users/Lenovo/Desktop/MARVEL-main/utils/model.py#L5-L46)	指针网络，用于动作选择

8.2 训练流程对应

组件	代码位置	说明
训练主循环	[driver.py#L145-L358](file:///C:/Users/Lenovo/Desktop/MARVEL-main/driver.py#L145-L358)	分布式训练循环
策略更新	[driver.py#L257-L266](file:///C:/Users/Lenovo/Desktop/MARVEL-main/driver.py#L257-L266)	SAC策略梯度更新
Q网络更新	[driver.py#L278-L296](file:///C:/Users/Lenovo/Desktop/MARVEL-main/driver.py#L278-L296)	双Q网络更新
温度参数更新	[driver.py#L298-L303](file:///C:/Users/Lenovo/Desktop/MARVEL-main/driver.py#L298-L303)	自适应温度更新

8.3 环境组件对应

组件	代码位置	说明
训练环境	[utils/env.py](file:///C:/Users/Lenovo/Desktop/MARVEL-main/utils/env.py)	使用 maps_medium 训练集
智能体类	[utils/agent.py](file:///C:/Users/Lenovo/Desktop/MARVEL-main/utils/agent.py)	智能体状态管理和决策

组件	代码位置	说明
节点管理	[utils/node_manager.py](file:///C:/Users/Lenovo/Desktop/MARVEL-main/utils/node_manager.py)	图节点管理和路径规划
传感器模块	[utils/sensor.py](file:///C:/Users/Lenovo/Desktop/MARVEL-main/utils/sensor.py)	FOV传感器模拟

报告生成时间：2026年7月23日